

COMP576 Homework 2

Jerry Yang

Department of Statistics

Rice University

Due Date: Nov 6, 2025

Contents

1	Question 1	3
1.1	1a	3
1.2	1a: First layer grayscale CIFAR-10 images	4
1.3	1b: Train Test Accuracy Lost Result (best result, others included in table below)	4
1.4	1b: Hyperparameter Tuning Summary	5
2	1c:Visualization	6
3	Question 2	8
4	Question 3	10
4.0.1	3a: RNN H-64 results	12
4.0.2	3b: LSTM H-64 Result	13
4.0.3	3c: GRU H-64 results	15
4.0.4	3d: RNN H-128 results	16
4.0.5	3e: LTSM H-128 results	18
4.0.6	3f: GRU H-128 results	19
4.1	Summary table of Training and Testing & Analysis	21

1 Question 1

1.1 1a

```
1  # Downloading the CIFAR-10 dataset
2  from torchvision import transforms, datasets
3  from torch.utils.data import DataLoader
4
5  if grayscale:
6      mean, std = [0.5], [0.5]
7      to_gray = [transforms.Grayscale(num_output_channels=1)]
8  else:
9      mean, std = [0.5, 0.5, 0.5], [0.5, 0.5, 0.5]
10     to_gray = []
11  transform = transforms.Compose(
12     to_gray + [
13         transforms.Resize((28, 28)),      # assignment wants 28x28
14         transforms.ToTensor(),
15         transforms.Normalize(mean=mean, std=std),]
16  )
17  train_dataset = datasets.CIFAR10(
18     root="./data", train=True, download=True, transform=transform
19  )
20  test_dataset = datasets.CIFAR10(
21     root="./data", train=False, download=True, transform=transform
22  )
23  train_loader = DataLoader(
24     train_dataset, batch_size=batch_size, shuffle=True,
25     num_workers=2, pin_memory=cuda
26  )
27  test_loader = DataLoader(
28     test_dataset, batch_size=batch_size, shuffle=False,
29     num_workers=2, pin_memory=cuda
30  )
```

1.2 1a: First layer grayscale CIFAR-10 images

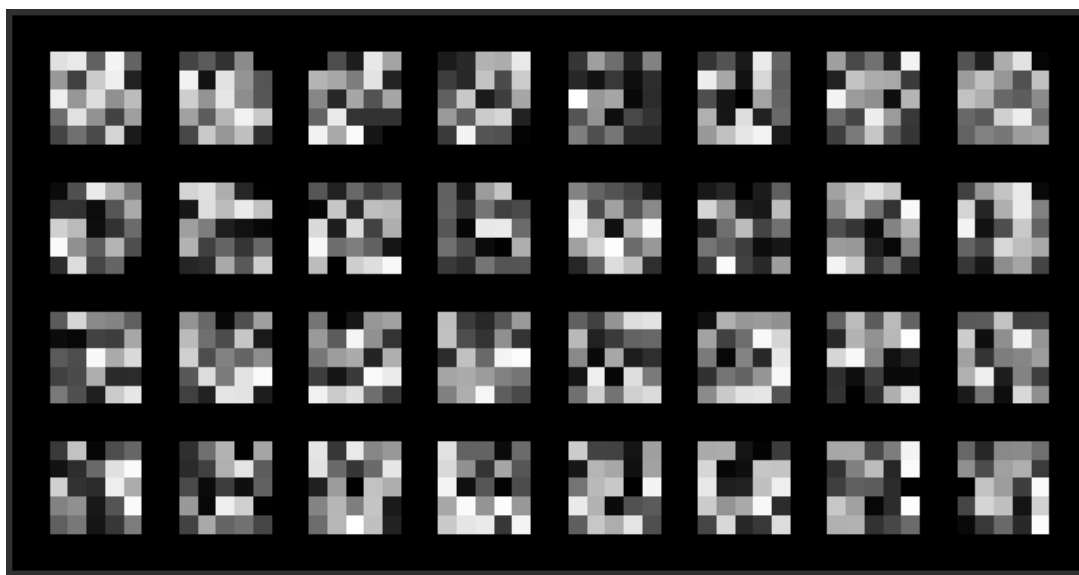


Figure 1: Sample Grayscale CIFAR-10 Images

1.3 1b: Train Test Accuracy Lost Result (best result, others included in table below)

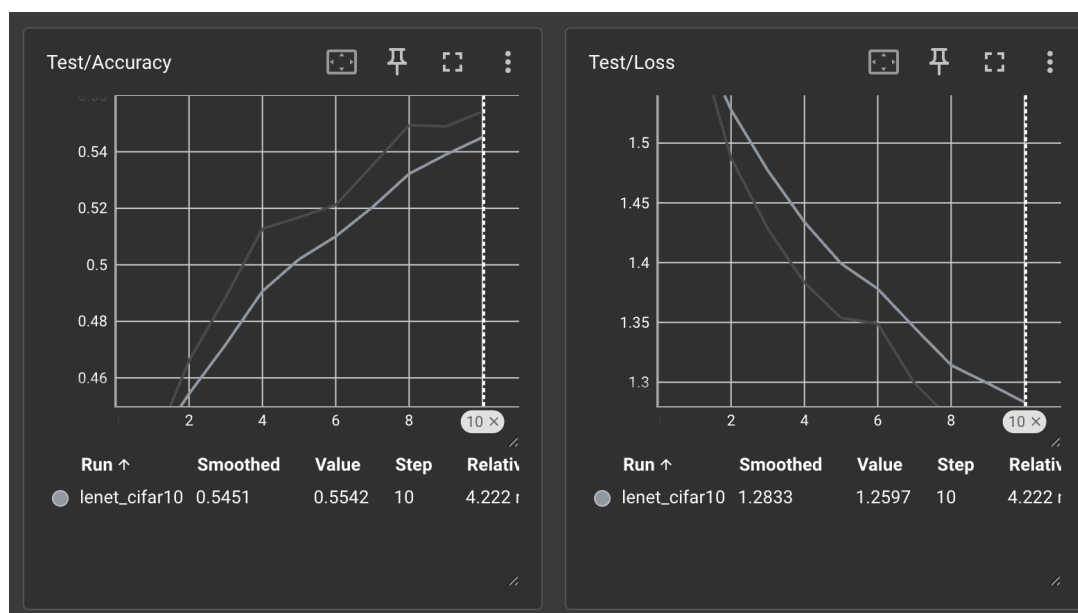


Figure 2: Accuracy Lost Result

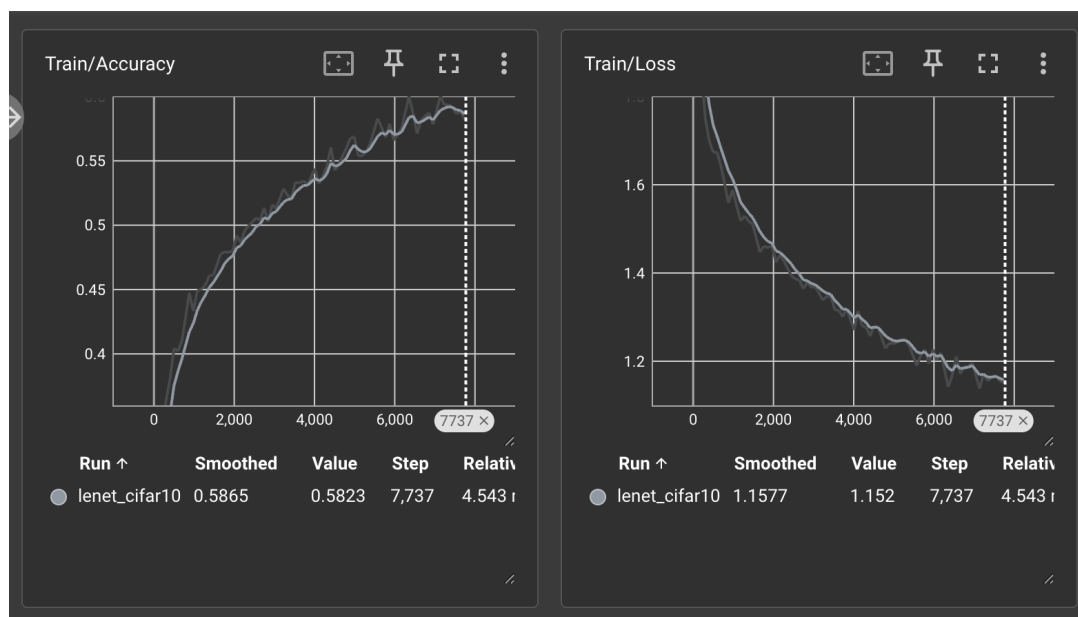


Figure 3: Accuracy Loss Result

1.4 1b: Hyperparameter Tuning Summary

Table 1: Hyperparameter Tuning Summary for LeNet-5 on CIFAR-10 (adam learn rate =0.01 best result)

ActFun	LR	Momentum	Train (%)	Test (%)
Adam	0.01	—	60.9	60.1
Adam	0.005	—	58.1	51.8
SGD	0.001	0.9	42.1	40.5
SGD	0.005	0.9	56.7	53.8

2 1c:Visualization

image filters

Conv1 Filters (Adam, LR=0.01)

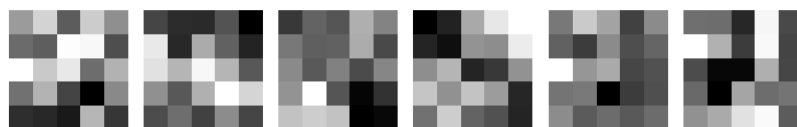


Figure 4: Adam w/ LR:0.01

Conv1 Filters (Adam, LR=0.005)

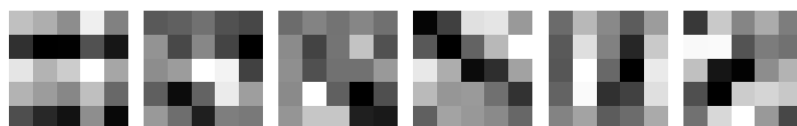


Figure 5: Adam w/ LR:0.05

Conv1 Filters (sgd, LR=0.01)

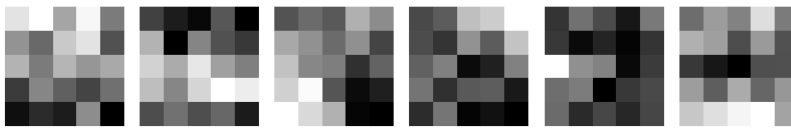


Figure 6: sgd w/ LR:0.01

Conv1 Filters (sgd, LR=0.05)

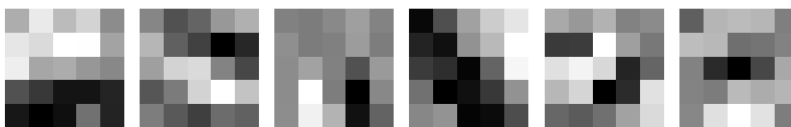


Figure 7: sgd w/ LR:0.05

statistics of the activations (best result only)

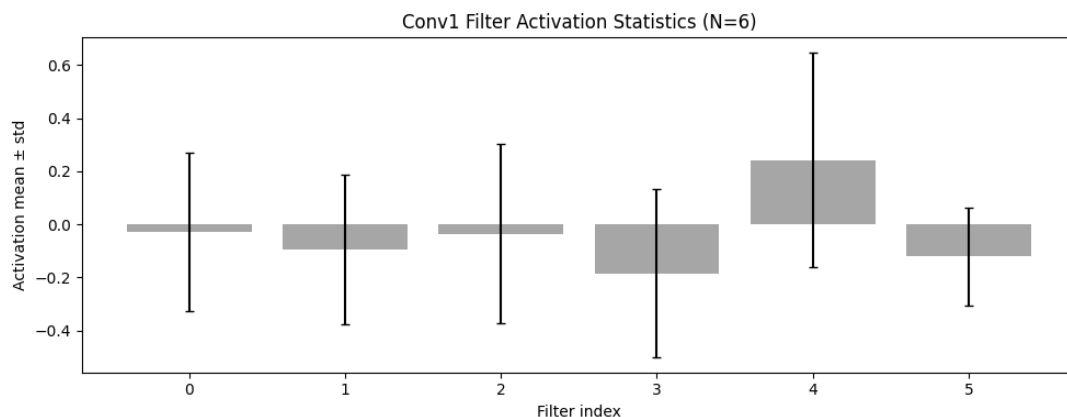


Figure 8: Per-filter activation mean and standard deviation for Conv1.

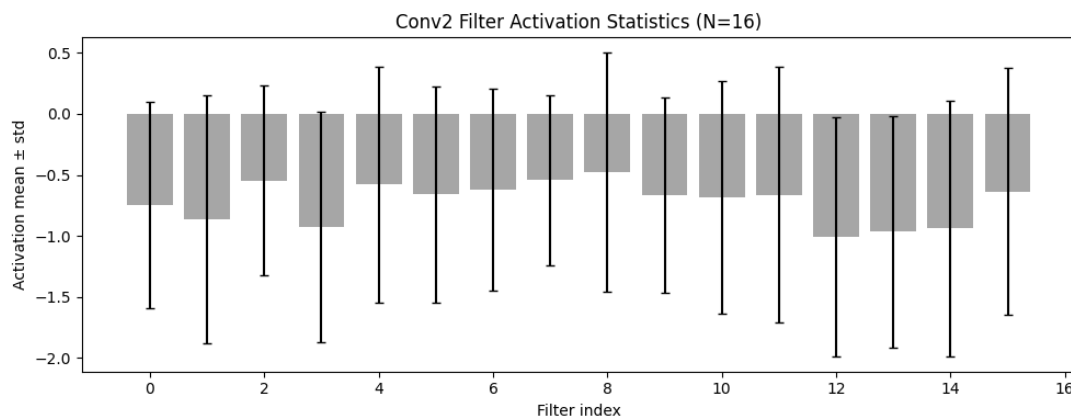


Figure 9: Per-filter activation mean and standard deviation for Conv2.

3 Question 2

Key Ideas of Zeiler & Fergus (2014)

Zeiler and Fergus (2014) proposed an approach to **visualize and understand convolutional neural networks (CNNs)**. Their work aimed to make CNNs less of a “black box” by showing what each layer learns and how features evolve during training. The key contributions can be summarized as follows:

- **Deconvolutional Network (DeconvNet) Framework:**

- Introduced a deconvnet that maps feature activations back to the image space, allowing direct visualization of what input patterns activate specific neurons.

- Each convolutional layer is paired with a corresponding deconvnet layer that performs inverse operations — unpooling (using pooling switches), ReLU, and convolution with transposed filters.
- This helps identify the visual structures (edges, textures, object parts) that each feature map responds to.

- **Layer-wise Feature Hierarchy:**

- Lower layers learn simple Gabor-like edge and color detectors.
- Intermediate layers detect object parts or textures (e.g., wheels, faces).
- Higher layers respond to complete object patterns (e.g., animals, vehicles).

- **Visualization as a Diagnostic Tool:**

- Deconvnet visualizations were used to analyze training dynamics and debug network performance.
- Helped identify ineffective or redundant filters and confirm that deeper layers capture semantically meaningful concepts.
- Occlusion experiments (blocking parts of an image) showed that prediction confidence drops when key object regions are hidden, proving spatial sensitivity.

- **Transfer Learning Insight:**

- Demonstrated that CNN features learned on large datasets (e.g., ImageNet) can generalize well to other tasks such as Caltech-101/256 classification.
- Highlighted how pre-trained convolutional layers capture reusable visual patterns.

In summary, the paper provided one of the first systematic frameworks for **interpreting and debugging CNNs**. The DeconvNet method gave engineers and researchers a practical way to visualize intermediate representations, understand the hierarchical nature of learned features, and improve network design through direct visual feedback.

4 Question 3

Code Listing 1: RNN Model Setup and Training Configuration

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Hyperparameter Configuration

input_size = 28          # each row of the image
sequence_length = 28     # number of rows
hidden_size = 128        # number of hidden units
num_layers = 1
num_classes = 10
batch_size = 64
num_epochs = 10
learning_rate = 0.001

#Preprocess
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(root='./data', train=True,
                                transform=transform, download=True)
test_dataset = datasets.MNIST(root='./data', train=False,
                                transform=transform, download=True)
```

```

train_loader = DataLoader(train_dataset , batch_size=batch_size ,
                           shuffle=True)
test_loader = DataLoader(test_dataset , batch_size=batch_size ,
                          shuffle=False)

```

RNN Model Definition

```

class RNNModel(nn.Module):
    def __init__(self , input_size , hidden_size , num_layers , num_classes):
        super(RNNModel, self).__init__()
        self.rnn = nn.RNN(input_size , hidden_size , num_layers ,
                            batch_first=True)
        self.fc = nn.Linear(hidden_size , num_classes)

    def forward(self , x):
        # x: (batch_size , 1, 28, 28) -> (batch_size , 28, 28)
        x = x.squeeze(1)
        out, _ = self.rnn(x)
        out = self.fc(out[:, -1, :]) # use last time step
        return out

```

Model, Loss, and Optimizer

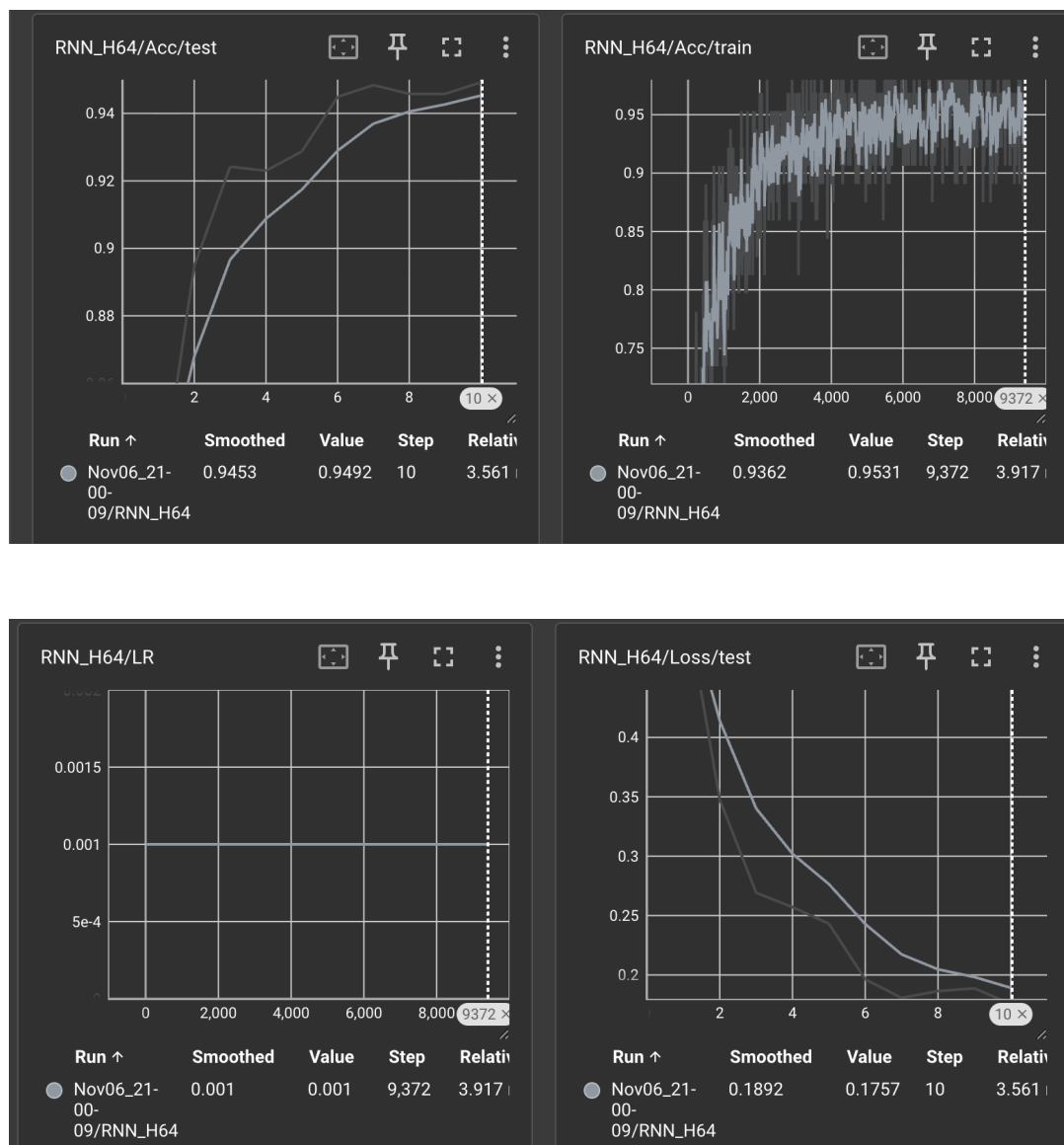
```

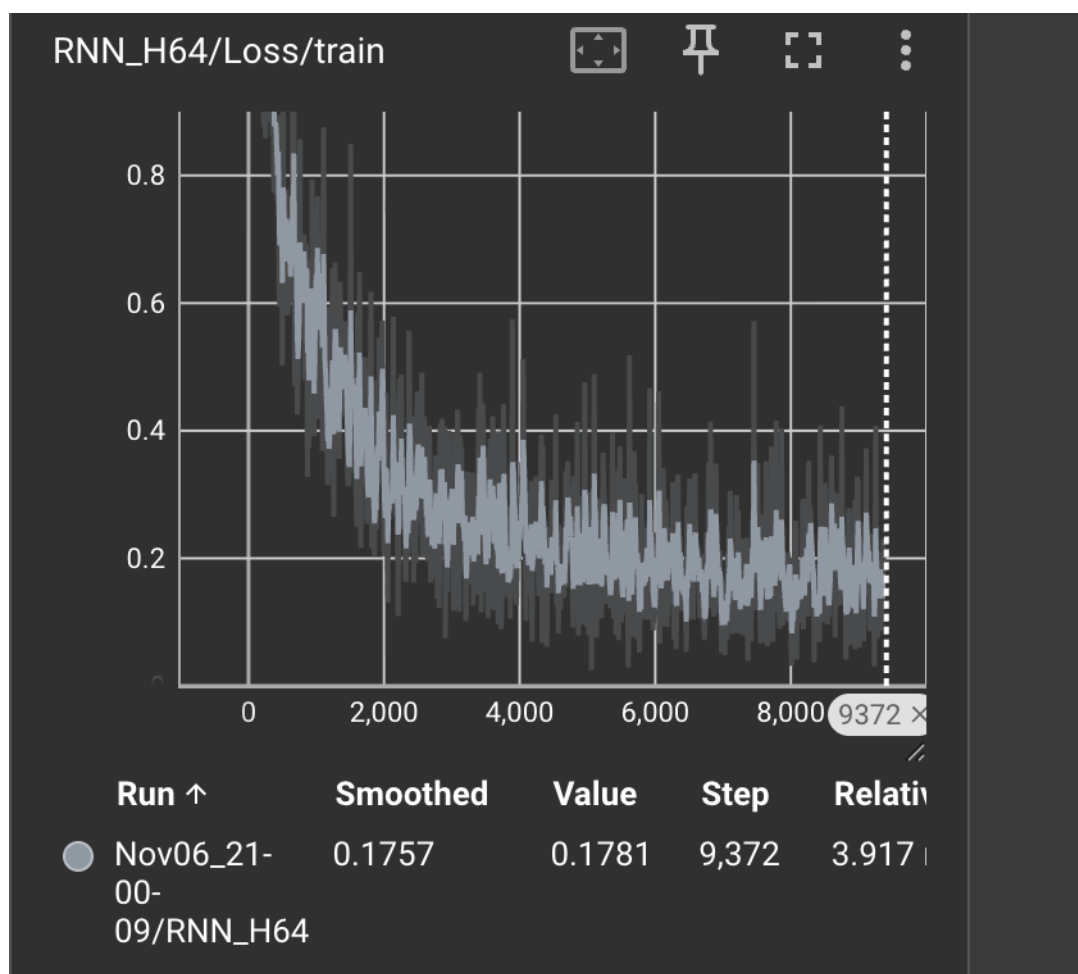
device = torch.device( cuda if torch.cuda.is_available() else cpu )
model = RNNModel(input_size , hidden_size , num_layers , num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters() , lr=learning_rate)

print(model)

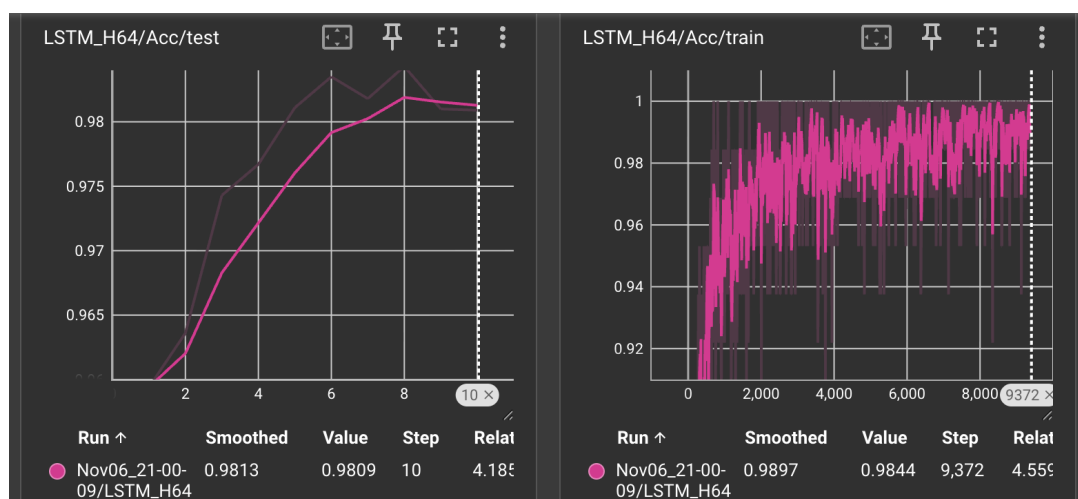
```

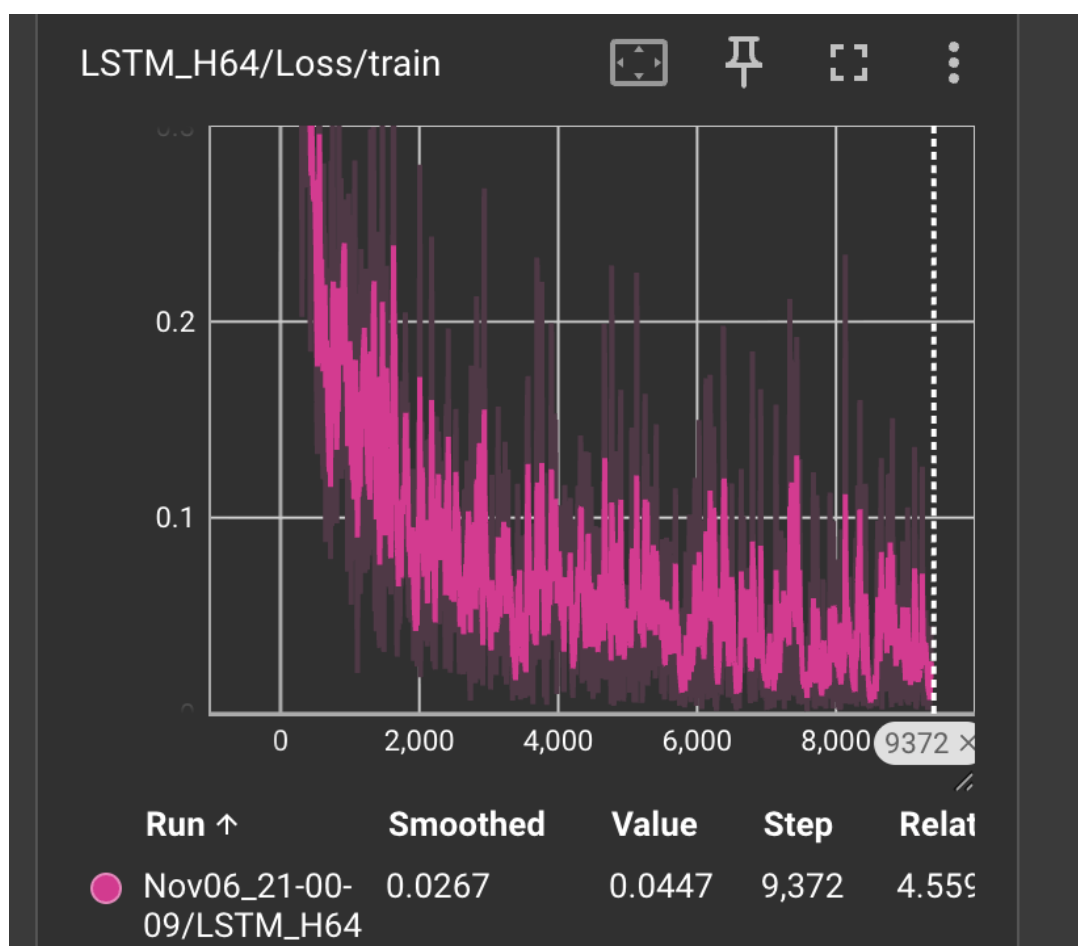
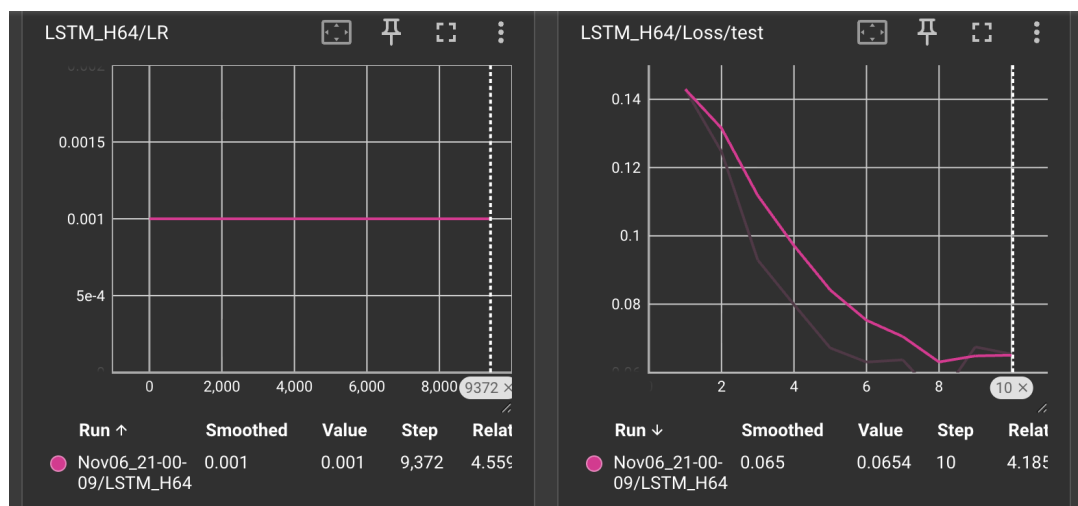
4.0.1 3a: RNN H-64 results



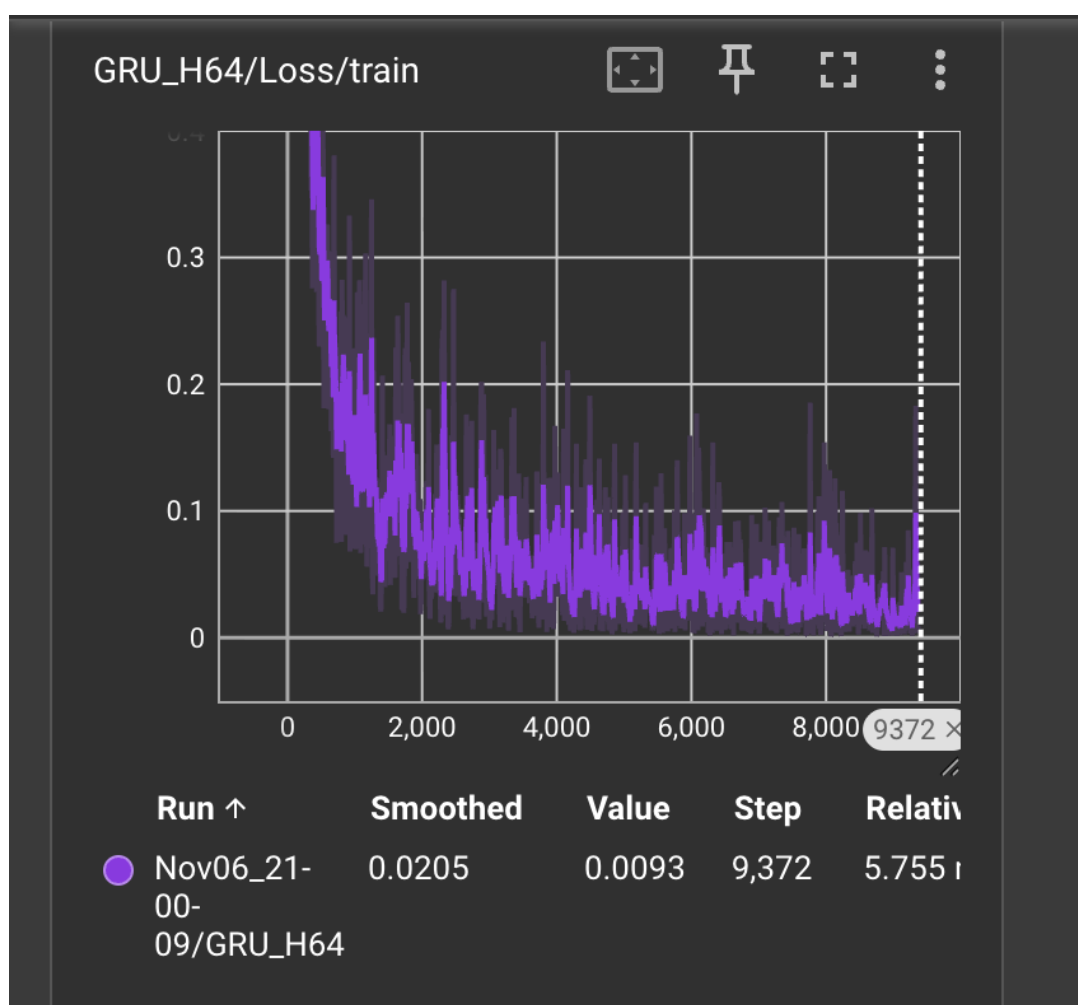
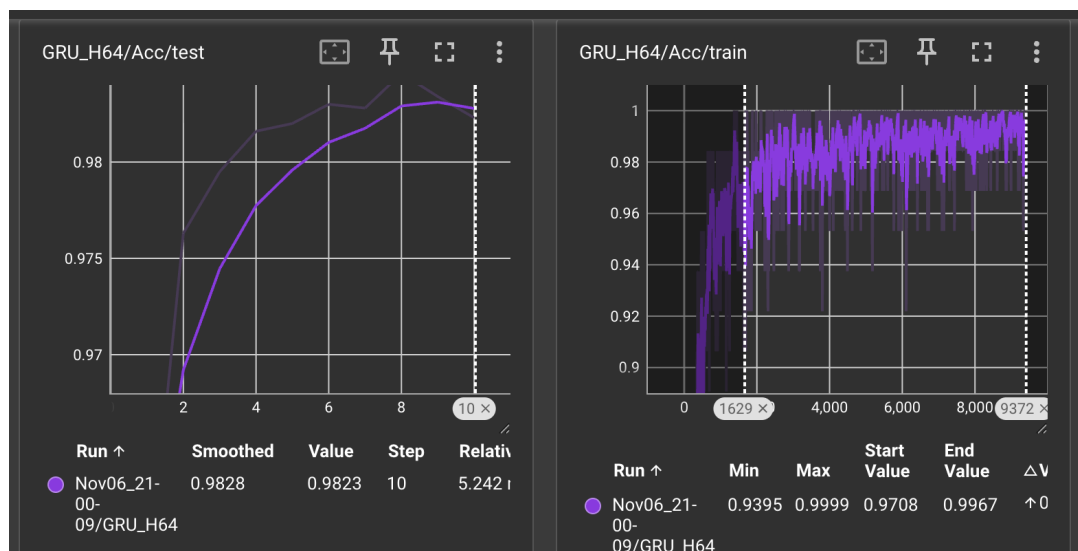


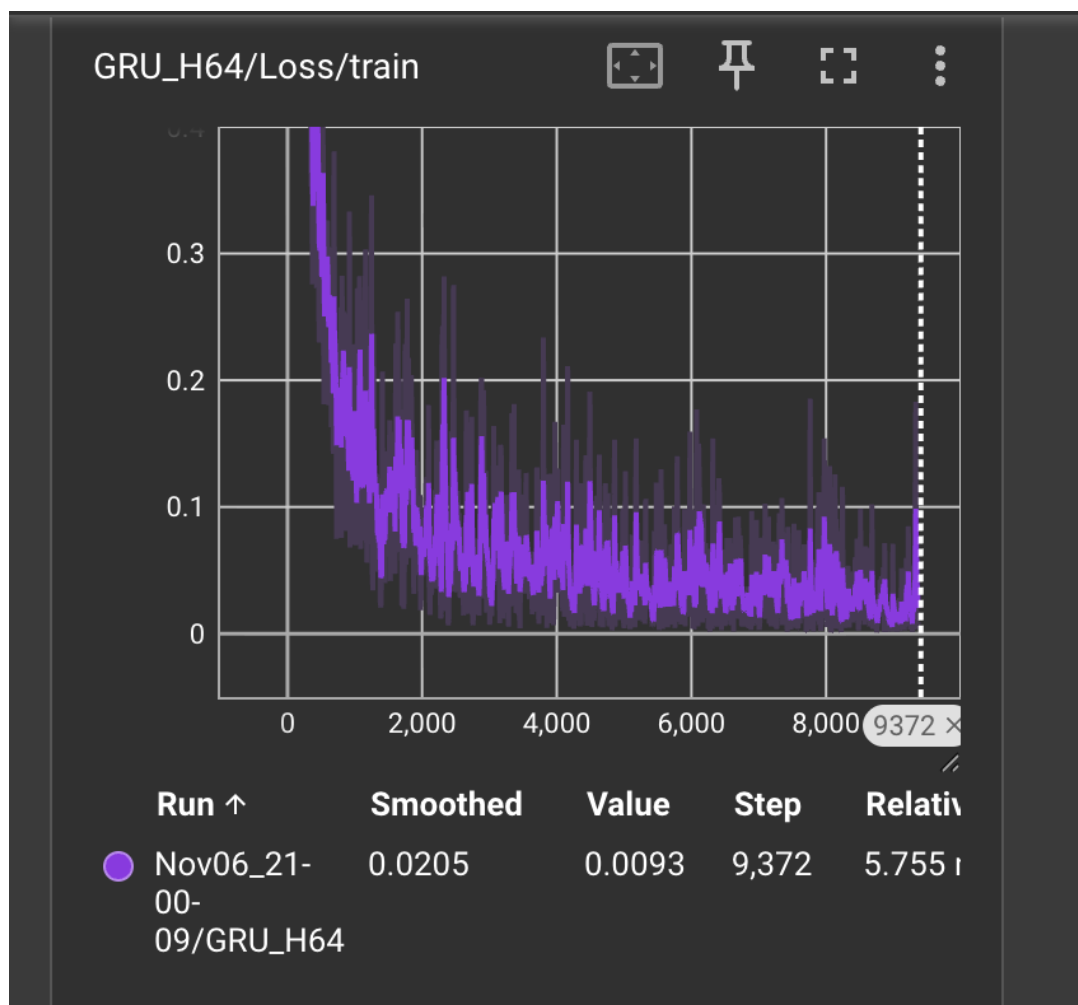
4.0.2 3b: LSTM H-64 Result



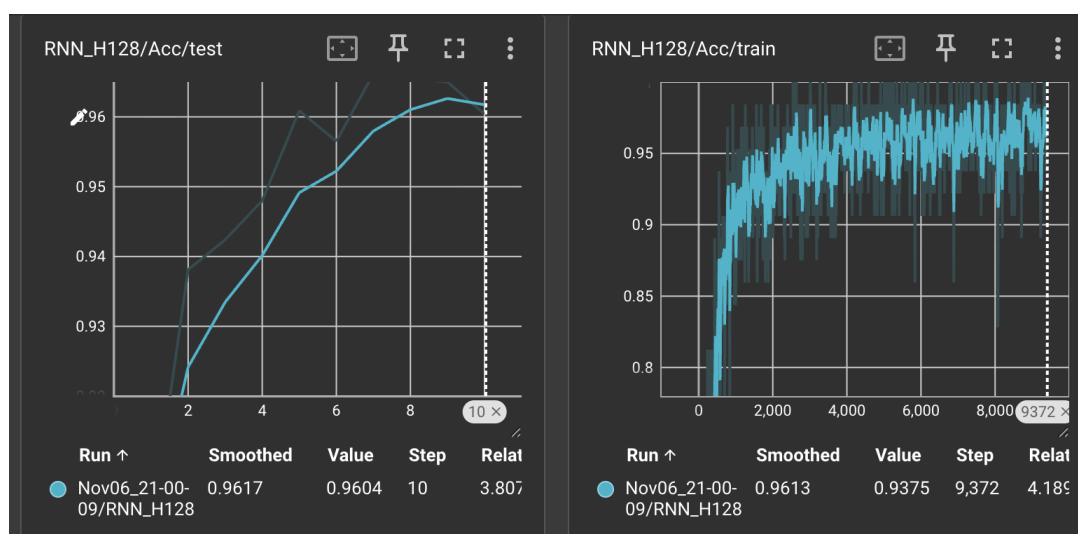


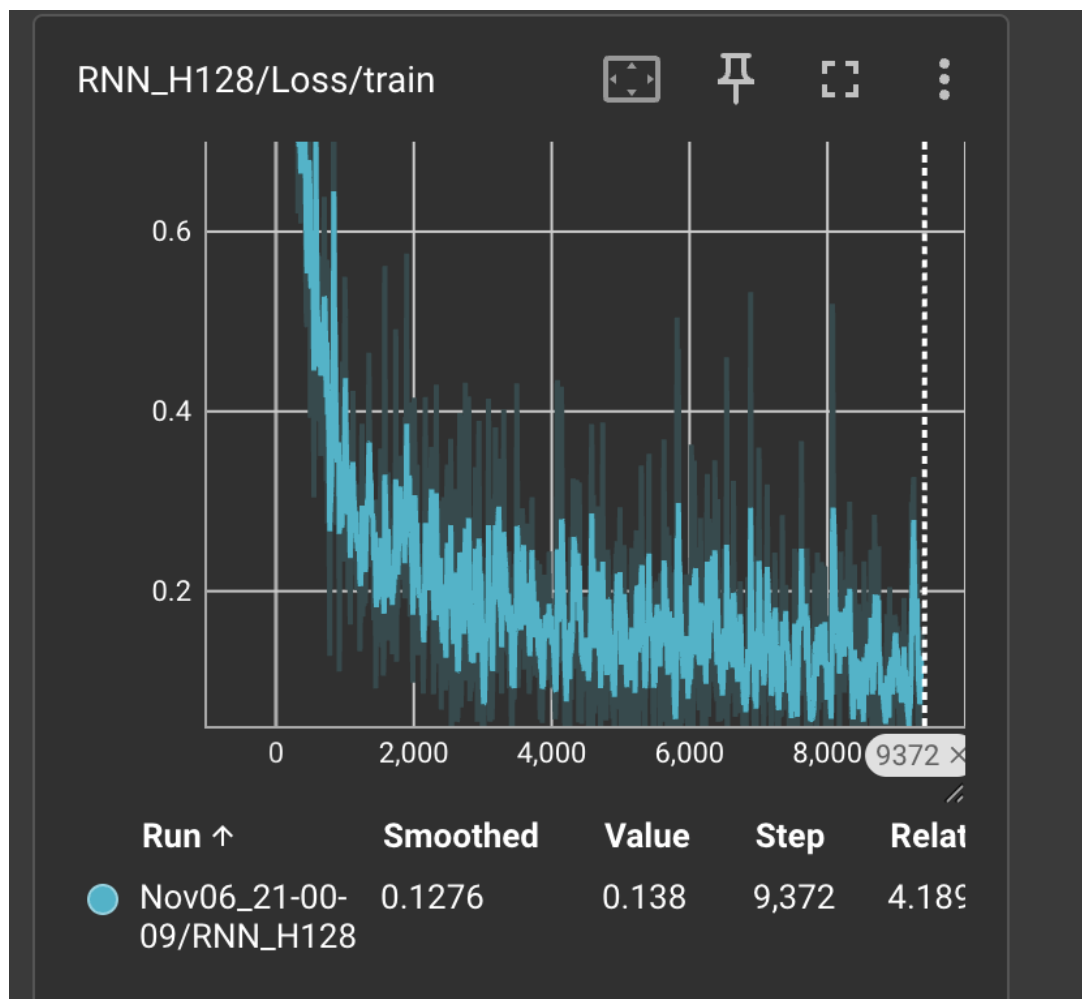
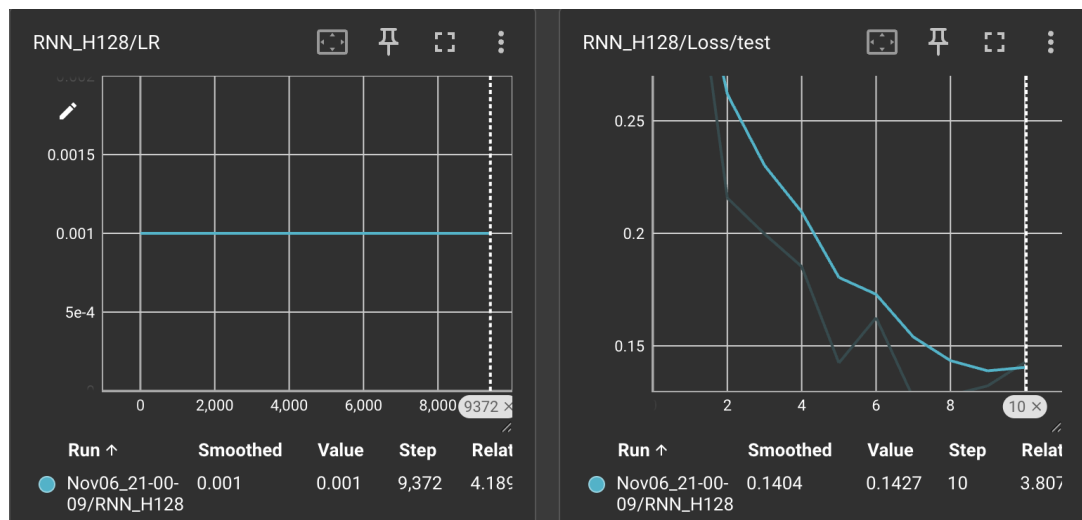
4.0.3 3c: GRU H-64 results



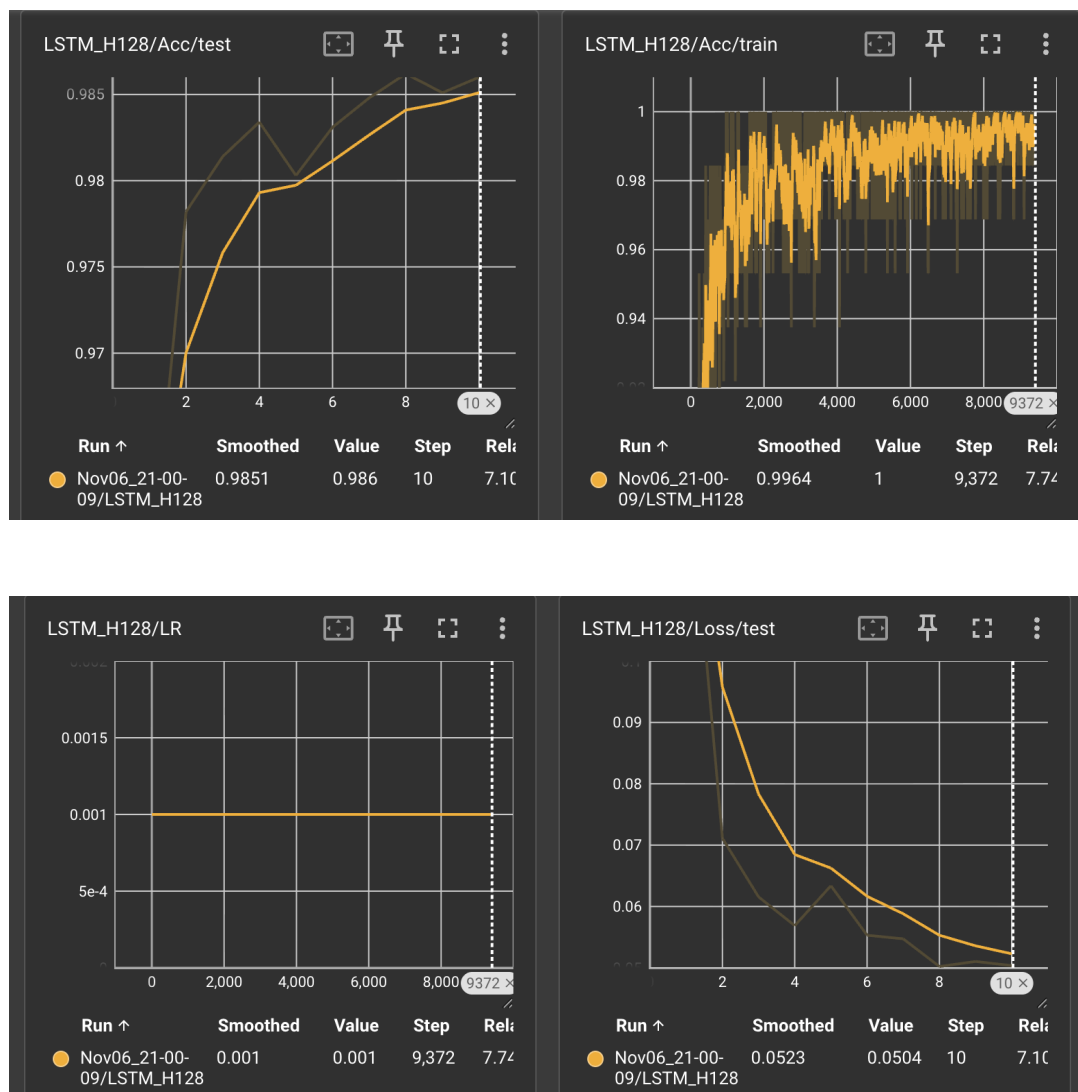


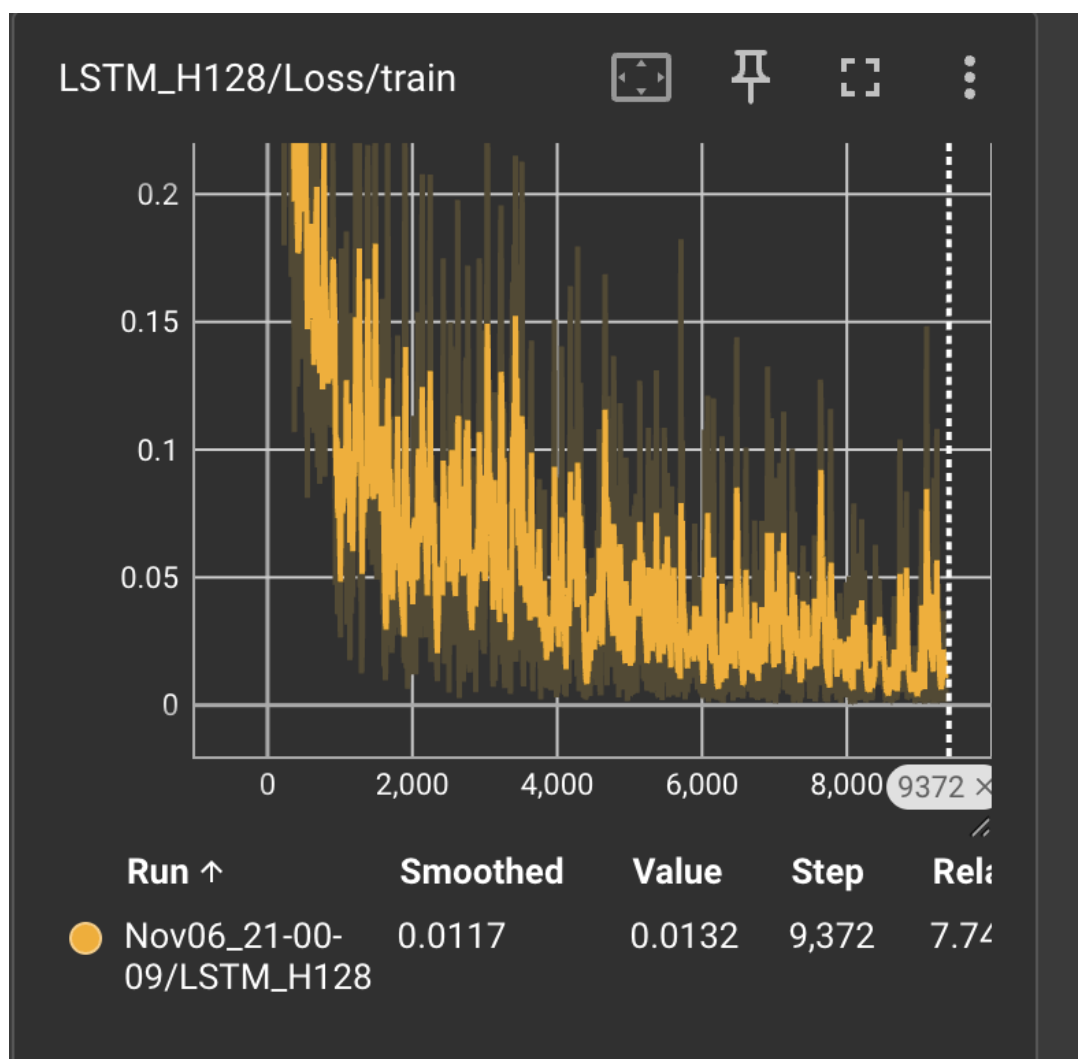
4.0.4 3d: RNN H-128 results



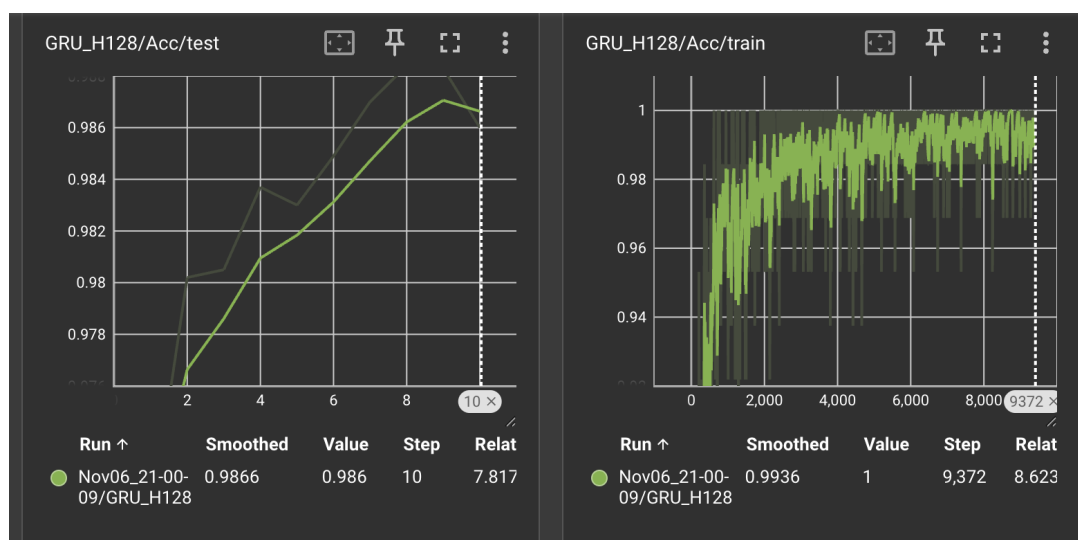


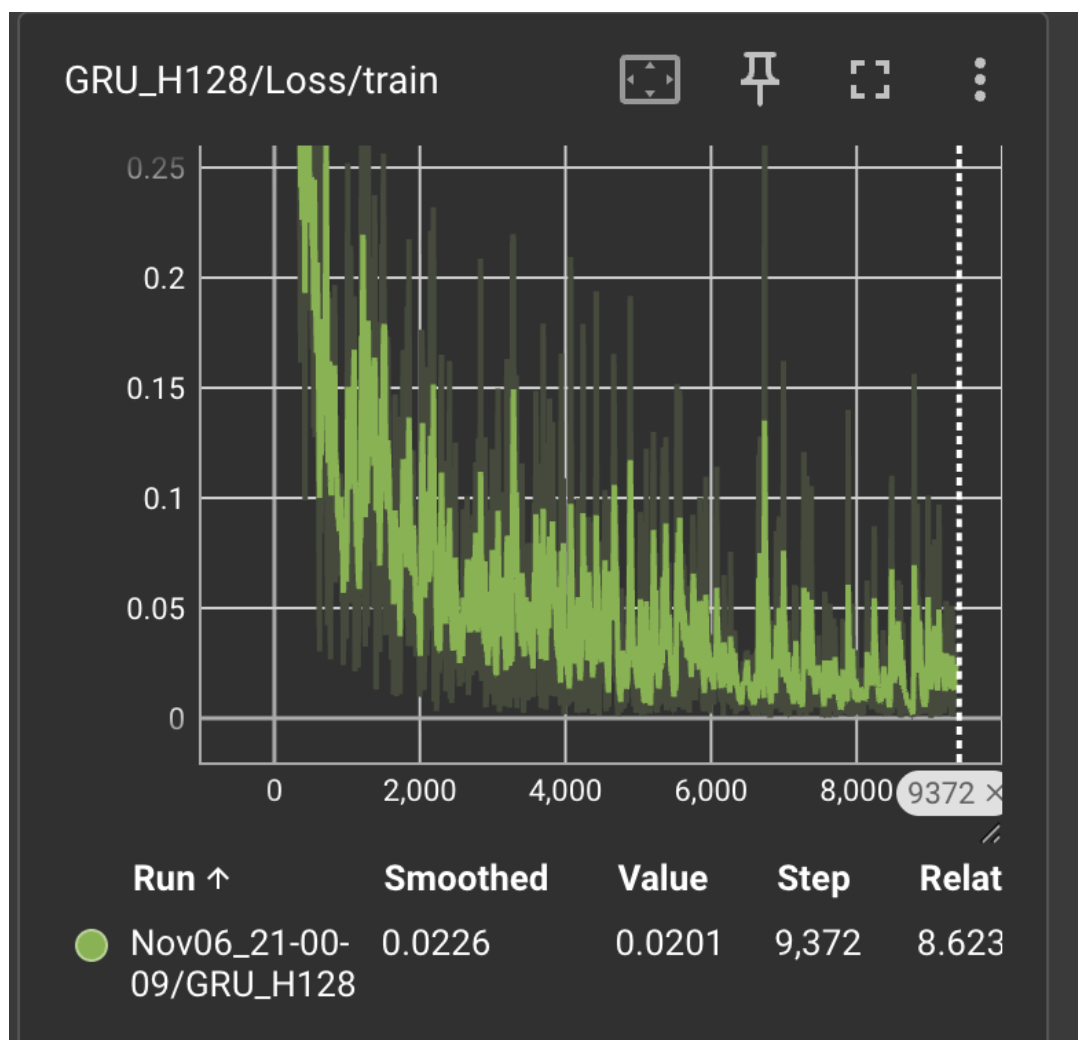
4.0.5 3e: LTSM H-128 results





4.0.6 3f: GRU H-128 results





4.1 Summary table of Training and Testing & Analysis

Table 2: Comparison of RNN, LSTM, and GRU models on MNIST with different hidden unit sizes.

Model	Hidden Units	Train Accuracy (%)	Test Accuracy (%)
RNN	64	95.31	93.62
RNN	128	96.04	96.13
LSTM	64	98.09	98.44
LSTM	128	99.64	98.6
GRU	64	99.62	98.28
GRU	128	99.36	98.66

Table 2 summarizes the training and testing accuracies for RNN, LSTM, and GRU architectures with different hidden unit sizes. The vanilla RNN achieved the lowest overall accuracy, reaching 95.31% (train) and 93.62% (test) with 64 hidden units. Increasing the hidden size to 128 improved the performance slightly to 96.04% and 96.13%, reflecting modest gains from increased model capacity.

In contrast, both LSTM and GRU significantly outperformed the standard RNN. The LSTM model reached 98.09% (train) and 98.44% (test) with 64 hidden units, and further improved to 99.64% and 98.60% when using 128 units. Similarly, the GRU model performed comparably, achieving 99.62% (train) and 98.28% (test) for 64 units, and 99.36% and 98.66% for 128 units. These results confirm that gated architectures (LSTM and GRU) can capture longer-term dependencies and mitigate vanishing gradient issues inherent in traditional RNNs. Between the two, GRU slightly outperforms LSTM in test accuracy while maintaining faster convergence, consistent with its more compact gating structure.

Effect of Hidden Units on Loss and Accuracy. Increasing the number of hidden units from 64 to 128 generally led to higher accuracy and smoother loss reduction for all models. This improvement arises because larger hidden representations enable the network to encode more complex temporal and spatial dependencies across image rows. However, the performance gain is relatively small (around 0.20.4%) and comes with increased training time and minor loss fluctuations, indicating the trade-off between capac-

ity and stability. For MNIST, which is a relatively simple dataset, the smaller hidden layer already achieves near-optimal accuracy, and adding more units yields diminishing returns. Notably, both LSTM and GRU scale more gracefully with hidden size compared to the vanilla RNN, maintaining stable convergence and lower loss variance across epochs.

Overall, LSTM and GRU consistently outperform the standard RNN in both convergence speed and final accuracy. While LSTM provides slightly better stability, GRU achieves comparable or better accuracy with fewer parameters, making it the most efficient architecture among the three for this task.

Question c: Comparison Between CNN and RNN

In this experiment, the CNN from Assignment 1 (LeNet-5 on CIFAR-10) achieved 58.23% training and 55.42% testing accuracy, while the RNN-based models trained on sequentialized 2828 MNIST inputs performed far better RNN: 96.04% / 96.13%, LSTM: 99.64% / 98.60%, GRU: 99.36% / 98.66%. The RNN family outperformed the CNN by over 40% in test accuracy. This gap mainly reflects architectural capacity and data structure: the shallow CNN struggled to capture complex spatial features, while RNNs effectively modeled sequential dependencies across pixel rows. Although CNNs typically excel in vision tasks when deeper, the simplified CNN here lacked the representational power of the RNN, LSTM, and GRU models, which achieved faster and more complete convergence.